

BUG VALIDATION DOCUMENT

Increase SQL Timeout in DataView API for OData Endpoint

1. Bug Summary & Context

Bug ID		Severity	Medium (Invasiveness: 3 - Low)
Title	Increase SQL Timeout in DataView API for OData Endpoint	Priority	P2
Environment	Production	Component	medrio.dataview.api
Test Type	Non-Functional / Configuration Validation	Test Activity	Customer Issue
Impacted Area	DataView API — OData Endpoint	Assignee	QA Team

Bug Description

The DataView API has a configurable SQL command timeout for Custom Views (default: 300 seconds, controlled by `CommonViewsSqlCommandTimeout`). However, this configurability was not available for OData endpoint calls. As a result, OData queries on large or complex datasets were not respecting a consistent, configurable timeout — creating an inconsistency between the two access patterns and causing customer-reported issues in Production.

Code Changes in Scope

- Startup.cs — DI registration updated to inject `MedrioSqlCommandTimeout`
- appsettings.json — New config key: `MedrioSqlCommandTimeout` (default: 300)
- MedrioDbContextFactory.cs — Constructor + `Database.SetCommandTimeout()` applied
- No schema changes, no API contract changes, no UI changes, no migration scripts

2. Structured Validation Approach

Step 1 — Reproduce & Confirm the Baseline

- Deploy build 1.0.0.75 from branch `feature/-increase_timeout` to QA/Staging
- Confirm `MedrioSqlCommandTimeout=300` is present in `appsettings.json`
- Identify a 'slow' OData entity — e.g., a large Study with 10,000+ Subjects — to simulate long-running queries
- Use Postman or cURL to fire: `GET {regionalPrefix}/api/odata/{entity}`

Step 2 — Isolate the Fix Layer

Layer	What to Check	How
Config Layer	<code>MedrioSqlCommandTimeout</code> exists in <code>appsettings.json</code>	Open <code>appsettings.json</code> or call config endpoint
DI / Startup	<code>MedrioDbContextFactory</code> registered with timeout injected	Review <code>Startup.cs</code> DI registration
DbContext Factory	<code>Database.SetCommandTimeout(300)</code> called on context creation	Add debug log or check <code>MedrioDbContextFactory.cs</code>
OData Endpoint	<code>GET /api/odata/{entity}</code> uses <code>MedrioDbContext</code> (not default context)	Trace request → DbContext instantiation in logs
SQL Server	Query on slow entity waits up to 300s before timeout	Check SQL Profiler or slow query logs

Step 3 — Test Data Strategy

- Fast entity: `Studies?$top=10` — should complete in < 2 seconds (sanity/regression)
- Slow entity: Subjects (large study) with complex `$filter` — must run > 60s for timeout tests
- Config permutations: `timeout = 60` (for timeout trigger), 300 (default), 600 (extended)

Step 4 — Environment Setup

- Base URL: `{regionalPrefix}` must be resolved from environment config (e.g., `https://qa.medrio.com`)
- Auth: Valid Bearer token obtained from auth service (scope: `dataview.read`)
- Log level: Set to `DEBUG` for `medrio.dataview.api` to capture `commandTimeout` log line
- Config reload: Restart DataView API service after any `appsettings.json` change

3. Test Plan

Scope	Validate that the configurable SQL timeout (MedrioSqlCommandTimeout) is correctly applied to all OData endpoint calls in the medrio.dataview.api service. Verify no regression to Custom Views or CustomDataViews.
Objective	Confirm bug fix: OData endpoint now uses MedrioSqlCommandTimeout (300s default). Ensuring behavior is consistent with Custom Views.
In Scope	- GET /api/odata/{entity} with \$filter, \$top, \$skip, \$orderby, \$expand - Config validation (appsettings.json) - Timeout trigger and error handling - Custom View regression
Out of Scope	- UI changes (none) - DB schema or migration - External API contracts - Performance benchmarking under full production load
Entry Criteria	- Build 1.0.0.75 deployed to Staging - MedrioSqlCommandTimeout=300 in appsettings.json - Auth token available - Slow-query test data
Exit Criteria	- All 22 test cases executed 0 P1/P2 defects open - Regression suite passing
Risks	- No slow-query test data available in QA — mitigate with WireMock delay stub - Config key naming may differ across environments — validate with DevOps - Concurrent test interference — run concurrency tests in isolated environment

4. Test Cases

22 test cases across 5 categories: Happy Path (5), Negative (5), Regression (4), Edge Cases (4), Security (4).

T C ID	Categ ory	Title / Scenario	Request	Expected Result	Act ual Res ult	P/ F	Notes
TC-001	Happy Path	Default timeout (300s) applied to OData endpoint	GET {prefix}/api/odata/Studies?\$top=10 Header: Authorization: Bearer <token>	HTTP 200 OK, response < 300s, logs show commandTimeout=300		—	Baseline smoke test
TC-002	Happy Path	OData GET with \$filter executes within timeout	GET {prefix}/api/odata/Subjects?\$filter=Status eq 'Active'&\$top=50	HTTP 200 OK with filtered records, query completes < 300s		—	Core fix validation
TC-003	Happy Path	OData GET with \$orderby returns sorted results	GET {prefix}/api/odata/Visits?\$orderby=VisitDate desc&\$top=20	HTTP 200 OK, records ordered by VisitDate descending		—	
TC-004	Happy Path	OData GET with \$skip and \$top returns paged results	GET {prefix}/api/odata/Events?\$top=25&\$skip=25	HTTP 200 OK, returns records 26–50, correct @odata.count		—	Pagination sanity check
TC-005	Happy Path	Config change from 300s to 600s reflects on OData	- Set MedrioSqlCommandTimeout=600 in appsettings.json - Restart service - GET {prefix}/api/odata/{entity}	New timeout 600s applied; logs confirm commandTimeout=600		—	Config-driven test

T C ID	Categ ory	Title / Scenario	Request	Expected Result	Act ual Res ult	P/ F	Notes
TC-006	Negative	Query exceeding configured timeout (60s) returns timeout error	1. Set timeout=60s 2. GET {prefix}/api/odata/{entity} with complex slow query 3. Wait >60s	HTTP 500 or 408, error body: 'Query execution timed out', no unhandled exception		—	Intentional timeout trigger
TC-007	Negative	Missing auth token returns 401	GET {prefix}/api/odata/Studies (no Authorization header)	HTTP 401 Unauthorized, no data leak		—	Security gate check
TC-008	Negative	Invalid entity name returns 404	GET {prefix}/api/odata/NonExistentEntity	HTTP 404 Not Found with descriptive error message		—	
TC-009	Negative	Malformed \$filter syntax returns 400	GET {prefix}/api/odata/Subjects?\$filter=INVALID_FILTER	HTTP 400 Bad Request, OData error structure returned		—	
TC-010	Negative	Config timeout set to 0 — fallback/error behavior	Set MedrioSqlCommandTimeout=0, restart, invoke slow query	Service handles gracefully (uses EF Core default or throws config error)		—	Edge of config range
TC-011	Regression	Custom Views timeout (300s default) not broken	Trigger existing Custom Views non-OData query (CommonViewsSqlCommandTimeout=300)	Behavior identical to pre-fix; Custom View timeout still respected		—	Critical regression — no cross-impact
TC-012	Regression	CustomDataView timeout (CustomViewsSqlCommandTimeout) unchanged	Invoke Custom Data View query, observe timeout behavior	CustomViewsSqlCommandTimeout still controls its own timeout independently		—	Parallel config isolation
TC-013	Regression	Existing OData GET \$top=10 still fast after config change	GET {prefix}/api/odata/Studies?\$top=10 after MedrioSqlCommandTimeout=600	HTTP 200 in < 5s; no unexpected latency introduced		—	Performance regression
TC-014	Regression	OData POST (if applicable) unaffected by timeout config	POST {prefix}/api/odata/{entity} with valid body	HTTP 201/200 as before; config change does not break POST		—	Check other HTTP methods
TC-015	Edge Case	Large dataset OData query completes within long timeout	Set timeout=600s, GET {prefix}/api/odata/{entity} (large entity, no \$top limit)	HTTP 200, full dataset returned within 600s		—	Long-running query happy path
TC-016	Edge Case	Concurrent OData requests each respect configured timeout	Send 10 parallel GET {prefix}/api/odata/{entity} requests	Each request independently applies commandTimeout=300; no shared state corruption		—	Thread safety / concurrency
TC-017	Edge Case	OData request with complex nested \$filter and \$expand	GET {prefix}/api/odata/Subjects?\$expand=Visits&\$filter=Age gt 18&\$orderby=Name	HTTP 200 with correct expanded payload, timeout applies to full query		—	Complex query type

TC ID	Category	Title / Scenario	Request	Expected Result	Actual Result	P/F	Notes
TC-018	Edge Case	Config file missing MedrioSqlCommandTimeout key — default applied	Remove MedrioSqlCommandTimeout from appsettings.json, restart service	Defaults to 300s (constructor default); service starts without error		—	Missing config resilience
TC-019	Security	Expired token rejected on OData endpoint	GET {prefix}/api/odata/Studies with expired Bearer token	HTTP 401 Unauthorized; no data returned		—	Auth validation
TC-020	Security	SQL injection in \$filter parameter is sanitized	GET {prefix}/api/odata/Subjects?\$filter=Name eq 'x'; DROP TABLE Subjects--	HTTP 400 Bad Request; no DB modification; query safely rejected		—	OData parser must sanitize
TC-021	Security	User A cannot access User B's OData data (RBAC)	Authenticate as User A, GET {prefix}/api/odata/{entity} scoped to User B's study	HTTP 403 Forbidden or filtered result showing only authorized records		—	Authorization scope check
TC-022	Security	Timeout config value not exposed in error response	Trigger timeout error deliberately, inspect HTTP response body	No internal config details (commandTimeout value, server paths) in response		—	Info disclosure check

5. Regression & Sign-Off Checklist

	Checklist Item	Owner	Linked TC
☐	Bug confirmed fixed: OData endpoint now uses MedrioSqlCommandTimeout (300s default)	QA Engineer	TC-001, TC-002
☐	Config change to 600s reflects on OData endpoint (logs confirm commandTimeout=600)	QA Engineer	TC-005
☐	Custom Views timeout behavior unchanged (CommonViewsSqlCommandTimeout still works)	QA Engineer	TC-011
☐	CustomDataView timeout (CustomViewsSqlCommandTimeout) still independent	QA Engineer	TC-012
☐	Query timeout error handled gracefully (HTTP 500/408, no crash, no stack trace exposure)	QA Engineer	TC-006
☐	Missing config key defaults to 300s — service starts without exception	QA Engineer	TC-018
☐	Concurrent OData requests do not share or corrupt DbContext timeout state	QA Engineer	TC-016
☐	SQL injection in \$filter is rejected; no DB modification	Security QA	TC-020
☐	RBAC: User A cannot see User B's OData data	Security QA	TC-021
☐	Timeout config value not exposed in API error response body	Security QA	TC-022
☐	All existing OData regression tests passing (no new failures)	Automation QA	TC-013, TC-014
☐	appsettings.json config key MedrioSqlCommandTimeout documented in release notes	QA Lead	N/A
☐	Deployed to Staging; behavior matches Dev test results	QA Lead	All TCs
☐	Production deployment sign-off confirmed by QA Lead	QA Lead	All TCs